Typescript

Created	@August 28, 2025 2:21 PM
Created by	Ⓛ LAKSHAY RAJ
Source	Learning's
Type	Note's
Reviewed	☐
Last edited time	@August 28, 2025 2:21 PM

▼ 🧠 Introduction & Setup

✅ Why Use TypeScript in Production?

Forget the "it's JavaScript with types" slogans. Here's the **real developer's reason**:

❌ Vanilla JS Pain	✅ TypeScript Solution
Hard to refactor large codebases	TS catches errors before you run code
Autocomplete is flaky/incomplete	TS gives **precise IntelliSense**
Bugs sneak in via dynamic typing	TS enforces **contracts** between modules
Docs go out of sync	Types **become documentation**
Working with 3rd party libs?	Type definitions tell you **exact usage**

> ✅ Bottom line: TypeScript makes your codebase scalable, maintainable, and less error-prone as it grows.

📦 Installing TypeScript (Global & Local)

🔷 Global Install (optional for CLI use)

```
npm install -g typescript
```

Check version:

```
tsc -v
```

🔷 Project-Local Install (recommended)

```
npm install --save-dev typescript
```

✅ This ensures all team members use the **same TS version**

✅ Essential for CI/CD, monorepos, or contributors

📁 Initialize a TypeScript Project

```
npx tsc --init
```

Creates a `tsconfig.json` file with default settings.

⚙️ `tsconfig.json` – Critical Fields Explained

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "strict": true,
    "jsx": "react-jsx",
    "allowJs": true,
    "paths": {
      "@components/*": ["src/components/*"],
      "@utils/*": ["src/utils/*"]
    },
    "baseUrl": "./",
```

```
    "esModuleInterop": true
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

🔍 Field Breakdown (Production Relevance)

Option	Why It Matters
`target: "ES2020"`	Emits modern JS; keeps code efficient
`module: "ESNext"`	Better tree-shaking in modern bundlers (Vite, Webpack, etc.)
`strict: true`	Enables all strict type checks (use it always ✅)
`jsx: "react-jsx"`	Supports React 17+ JSX transform
`allowJs: true`	Allows `.js` files in TS project (useful for gradual migration)
`esModuleInterop: true`	Fixes `import x from 'commonjs-lib'` errors
`baseUrl`	Required for `paths` to work
`paths`	Create clean import aliases (e.g., `@components/Button`)
`include` / `exclude`	Controls what TS scans (reduces compile time)

✅ Folder Structure for TS Projects (React or Node.js)

```
project/
├── src/
│   ├── components/
│   ├── utils/
│   ├── types/
│   └── index.ts
├── tsconfig.json
└── package.json
```

✅ Keep all logic inside `src/`

✅ Add shared types inside `types/` folder

✅ Avoid putting TS files in root for cleaner scaling

🔐 Best Practices for Setup

Tip	Why
Enable `strict: true` from day 1	Prevents subtle bugs later
Use `paths` for cleaner imports	Improves DX and refactoring
Don't ignore `tsconfig` warnings	Fix root cause, don't silence types
Use `tsconfig.build.json` for library mode	Separate build config = clean builds
Combine with ESLint + Prettier	Format + lint + type-check together for real productivity

---

✅ Minimal React Setup (Vite)

```
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
```

Vite + TypeScript = blazing fast + DX optimized

✅ Minimal Node.js Setup (Express)

```
npm init -y
npm install express
npm install -D typescript @types/express ts-node-dev
```

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2020",
```

```
    "module": "CommonJS",
    "strict": true,
    "outDir": "dist"
  }
}
```

Add `ts-node-dev` script:

```
"scripts": {
  "dev": "ts-node-dev src/index.ts"
}
```

⚠️ Production Warnings

❌ Mistake	✅ Fix
Leaving `strict` disabled	Turn it on or suffer during scale-up
Not using type aliases for paths	Codebase gets spaghetti imports
Not installing `@types/*` packages	Leads to missing IntelliSense and types
Using `.js` files randomly in TS	Use `allowJs` only during migration, not forever
Letting `any` creep in everywhere	Use `unknown`, `never`, or proper types

---

📌 Summary Cheatsheet

Feature	Config
Project Init	`npx tsc --init`
React JSX	`"jsx": "react-jsx"`
Strict Types	`"strict": true`
Aliases	`baseUrl` + `paths`
Compile Target	`"target": "ES2020"`
Module System	`"module": "ESNext"` or `"CommonJS"`
JS Interop	`"esModuleInterop": true`

Feature	Config
Dev Setup	Use `ts-node-dev` or Vite

▼ 📦 Basic Types (The Production-Friendly Way)

✅ 1. Primitive Types

Type	Example	Notes
`string`	`"hello"`	Use for text
`number`	`42`, `3.14`	Covers all JS numbers
`boolean`	`true`, `false`	Logic values

```
let title: string = "TS Mastery";
let views: number = 12000;
let isPublished: boolean = true;
```

✅ TS usually infers these types, but you can annotate them when needed.

✅ 2. Arrays

There are **two syntaxes** – use either, stay consistent.

```
const scores: number[] = [100, 95, 80];
const names: Array<string> = ["Alice", "Bob"];
```

> 💡 Prefer number[] or string[] in most React codebases. It's cleaner.

✅ Arrays can be typed with objects too:

```
type User = { id: number; name: string };
const users: User[] = [{ id: 1, name: "Shakil" }];
```

✅ 3. Tuples (Ordered fixed-length arrays)

```
let response: [number, string] = [200, "OK"];
```

✅ Use when order matters (e.g., `[statusCode, message]`)

❌ Don't abuse for readability-heavy data — prefer objects

⚠️ 4. Enums – Legacy Pattern (use sparingly)

```
enum Role {
  ADMIN,
  USER,
  GUEST
}

const userRole: Role = Role.ADMIN;
```

❌ Don't Use in Modern Codebases Unless:

- You're dealing with **older APIs**
- You need **constant mapping**

✅ Prefer Union Types:

```
type Role = "admin" | "user" | "guest";
const role: Role = "admin";
```

✅ Better intellisense, smaller bundle, easier to refactor

🧠 5. Special Types (🔥 Important)

🔷 `any`: Escape hatch (⚠️ avoid if possible)

```
let data: any = "Could be anything";
```

> ❌ Kills all type safety. You're back to vanilla JS.

🔷 `unknown`: Safer version of `any`

```
let input: unknown;

if (typeof input === "string") {
  input.toUpperCase(); // ✅ Type narrowed
}
```

✅ Great for user input, external API responses

✅ Forces **type checking before usage**

🔷 `never`: Type that never occurs

```
function crash(): never {
  throw new Error("This always throws");
}
```

✅ Use to type functions that **never return**

✅ Also used in exhaustive switch checks

🔷 `void`: No return value

```
function log(msg: string): void {
  console.log(msg);
}
```

✅ Functions that **only perform side effects**

✅ In React event handlers: `onClick: () => void`

⚠️ Production Warnings & Best Practices

❌ Mistake	✅ Best Practice
Overusing `any`	Use `unknown` if needed, or define a type
Using `enum` everywhere	Prefer union string literals
Assuming `never` = error	Use for functions that never return or in switch checks
Forgetting `void` return	Causes confusion in event handlers
Forgetting to narrow `unknown`	Use `typeof`, `instanceof`, or custom checks

🧪 Summary Cheatsheet

Type	Purpose
`string`, `number`, `boolean`	Basic primitives
`number[]`	Array of values
`[string, number]`	Tuple – fixed, ordered values
`"a" | "b"`	Union literals (great for role/status)
`any`	Unsafe escape (avoid!)
`unknown`	Safer input – must narrow
`never`	Function never completes (e.g. throw)
`void`	No return value

📌 Real Usage Examples

```
function getStatus(code: number): [number, string] {
  if (code === 200) return [200, "OK"];
  return [500, "Server Error"];
}

type Theme = "light" | "dark";
let currentTheme: Theme = "light";

function handleInput(input: unknown): void {
```

```
  if (typeof input === "string") {
    console.log("Length:", input.length);
  }
}
```

▼ 🔧 Type Aliases & Interfaces

✅ 1. Type Alias (`type`)

Use `type` when defining:

- **Objects**
- **Primitives**
- **Unions**
- **Tuples**
- **Function signatures**
- **Any complex type expression**

```
type User = {
  id: number;
  name: string;
  isActive: boolean;
};
```

✅ Flexible

✅ Can be used for non-object shapes too

```
type ID = number | string;
type Point = [number, number];
```

✅ 2. Interface

Use `interface` to define **object shape only**.

```
interface User {
  id: number;
  name: string;
  isActive: boolean;
}
```

✅ Extendable

✅ Ideal for class-based code or React props

✅ More optimized internally by TS (marginal)

🔍 3. Type vs Interface — What's the Difference?

Feature	`interface`	`type`
Can extend?	✅ Yes (via `extends`)	✅ Yes (via `&`)
Merge-able?	✅ Yes (declaration merging)	❌ No
Union/tuple support	❌ No	✅ Yes
React props style	✅ Common	✅ Common
Function signature	❌ No	✅ Yes
Preferred when?	Public APIs, libs, class shapes	Union types, composition, flexibility

🔶 Example:

```
// Interface can be merged
interface User {
  name: string;
}
interface User {
  age: number;
}
// Result: { name: string; age: number }
```

> ⚠️ type declarations don't merge — you'll get a duplicate error if redeclared.

🧩 4. Extending Types (Inheritance)

🔷 Extending `interface` (OOP style)

```
interface Person {
  name: string;
}

interface Employee extends Person {
  employeeId: number;
}
```

🔷 Extending `type` (using `&`)

```
type Person = {
  name: string;
};

type Employee = Person & {
  employeeId: number;
};
```

> ✅ Both result in the same final shape. Use whatever fits the project style or team standard.

✅ Real-World Usage Patterns

Scenario	Use
React props	`interface` or `type`, both valid
Union types	`type`

Scenario	Use
Function signatures	`type`
Public SDKs/libs	`interface` (merging is useful)
Extending multiple types	`type` via `&` is cleaner
Working with `class`	`interface` is idiomatic

✅ Functions with Type Aliases

```
type Greet = (name: string) => string;

const greet: Greet = (name) => `Hello, ${name}`;
```

✅ You can't do this directly with `interface`

⚠️ Production Warnings

❌ Pitfall	✅ Best Practice
Re-declaring `type`	❌ Throws error (no merge support)
Using `type` for class shapes	Prefer `interface` for consistency
Extending `type` with large unions	Can get hard to debug — break into smaller types
Merging `interface` for unrelated things	Avoid — makes it hard to trace final shape

🧪 Summary Cheatsheet

Feature	`type`	`interface`
Object types	✅	✅
Function types	✅	❌
Union/tuple support	✅	❌
Extendable	✅ (`&`)	✅ (`extends`)

Feature	`type`	`interface`
Merge-able	❌	✅
Recommended for libraries	❌	✅
Ideal for OOP/classes	❌	✅

✨ Real Example: React Props

```
interface ButtonProps {
  label: string;
  onClick: () => void;
}

const Button = ({ label, onClick }: ButtonProps) => (
  <button onClick={onClick}>{label}</button>
);
```

✅ Interfaces are commonly used for clarity in React codebases

✅ But `type` is also fine if you're using composition heavily

📌 Summary

- Use `type` for **flexibility** (unions, functions, tuples)
- Use `interface` for **structure** (especially objects/classes)
- Don't overthink — both compile to the same JavaScript

▼ 🧩 Functions in TypeScript

✅ 1. Typing Parameters & Return Types

```
function add(a: number, b: number): number {
  return a + b;
}
```

- Parameter types go **inside** `()`
- Return type goes **after** `):`

```
function logMessage(msg: string): void {
  console.log(msg);
}
```

> ✅ void is for functions with no return value (e.g. side effects)

🧪 Best Practice

Always **explicitly type return value** in:

- Shared utility functions
- API functions
- Publicly exposed library methods

> 💡 TS can infer return types, but explicit types help during refactors

---

🧩 2. Optional & Default Parameters

🔶 Optional (`?`):

```
function greet(name?: string): string {
  return `Hello, ${name ?? "Guest"}`;
}
```

✅ Parameter may be `undefined`

🔶 Default:

```
function greet(name: string = "Guest"): string {
  return `Hello, ${name}`;
```

```
}
```

✅ Cleaner than optional when you want a fallback value

🧠 3. Function Type Aliases

Create reusable signatures:

```
type GreetFn = (name: string) => string;

const greet: GreetFn = (name) => `Hello, ${name}`;
```

✅ Great for callbacks, event handlers, hooks

> 💡 Helps you reuse logic signature across components

🔥 4. Arrow Function Typing

There are **two ways** to type arrow functions:

✅ Best Practice (type the function variable):

```
const multiply: (a: number, b: number) => number = (a, b) => a * b;
```

✅ Cleaner for inline code, auto-infers param types

❌ Anti-pattern: Typing inside arrow args

```
// ❌ Avoid this
const add = (a: number, b: number): number => a + b;
```

> Not wrong, but less readable & DRY for callbacks or props

🧩 5. Call Signatures in Interfaces

Perfect for **functions as objects** or **prop typing** in React.

```
interface Logger {
  (message: string): void; // call signature
}

const log: Logger = (msg) => console.log(msg);
```

✅ Real use case: React component props

```
interface ButtonProps {
  onClick: () => void;
}
```

✅ Cleaner than declaring a separate type alias

✅ Autocompletes in JSX

🛡️ Bonus: Rest Parameters + Return Union Types

```
function join(...args: string[]): string {
  return args.join(", ");
}
```

```
function getStatus(code: number): string | null {
  if (code === 200) return "OK";
  return null;
}
```

✅ Use union return types for expressive return behavior

✅ `never` return type = function that always throws

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Not typing return values	Leads to unexpected inference issues
Using optional param before required	Always put optional **after** required ones
Not reusing function types	Use type aliases or call signatures for clarity
Overusing `any` in callback types	Define `event: MouseEvent` etc. properly

✅ Real Usage Examples

🔷 API Utility Function

```
type FetchData = (url: string) => Promise<any>;

const fetchData: FetchData = async (url) => {
  const res = await fetch(url);
  return res.json();
};
```

🔷 React Props with Function

```
interface ModalProps {
  onClose: () => void;
}

const Modal = ({ onClose }: ModalProps) => (
  <button onClick={onClose}>Close</button>
);
```

🧪 Summary Cheatsheet

Feature	Syntax
Function type	`(a: T, b: T) => ReturnType`
Arrow function typing	`const fn: (a: T) => R = (a) => ...`

Feature	Syntax
Optional param	`name?: string`
Default param	`name: string = "Guest"`
Call signature in interface	`interface MyFn { (arg: T): R }`
Void return	`(): void`
Never return	`(): never`

▼ 🛠️ Objects & Structural Typing

✅ 1. Strict Object Typing

Define exact object shape using either `type` or `interface`.

```
type User = {
  id: number;
  name: string;
  isActive: boolean;
};

const user: User = {
  id: 1,
  name: "Shakil",
  isActive: true,
};
```

✅ TypeScript enforces that `user` has **exactly** those properties

❌ Extra or missing fields will cause an error

🧠 Structural Typing (aka "Duck Typing")

TypeScript uses **structural typing**, meaning:

> “If it walks like a duck and quacks like a duck, it's a duck.”

```
type Point = { x: number; y: number };

function logPoint(p: Point) {
  console.log(p.x, p.y);
}

const pointLike = { x: 10, y: 20, z: 30 };
logPoint(pointLike); // ✅ OK — extra fields are fine if not directly assigned
```

⚠️ Structural typing **does allow extra fields**, unless directly assigned to a typed variable.

❌ Direct Assignment = Exact Match Required

```
const p: Point = { x: 10, y: 20, z: 30 }; // ❌ Error: 'z' is not allowed
```

✅ Use `as Point` if you want to allow loose objects

🧩 2. Optional Properties (`?`)

```
type User = {
  id: number;
  name?: string;
};

const user: User = { id: 1 }; // ✅ name is optional
```

- Great for partial data (e.g. `UserPreview`, `Partial<T>`)
- Combine with default values or conditionals

🔐 3. Readonly Properties

```
type Config = {
  readonly apiKey: string;
};

const cfg: Config = { apiKey: "123" };
cfg.apiKey = "456"; // ❌ Cannot assign to readonly property
```

✅ Use for immutable constants, env config, props

> 💡 React's props are readonly by default

🔑 4. Index Signatures (`[key: string]: valueType`)

Used for **dynamic key-value objects** like dictionaries:

```
type StringMap = {
  [key: string]: string;
};

const colors: StringMap = {
  primary: "#000",
  secondary: "#fff",
  warning: "#f00",
};
```

✅ Useful when keys are not known ahead of time

🔷 Indexed Access (Read only specific keys)

```
type StatusMap = {
  success: string;
  error: string;
  [key: string]: string;
};
```

```
const status: StatusMap = {
  success: "OK",
  error: "Failed",
  loading: "Loading...",
};
```

> ✅ You can have fixed keys + flexible keys together

⚠️ Pitfall: Index Signature Overrides All

```
type BadExample = {
  name: string;
  [key: string]: number; // ❌ 'name' can't be string now
};
```

> ❌ Don’t mix fixed and index types with mismatched types

✅ Match the value types of all fixed fields with the index signature

---

🧪 Bonus: Mapped Types (Powerful combo)

```
type Flags = {
  darkMode: boolean;
  mobile: boolean;
};

type ReadonlyFlags = {
  readonly [K in keyof Flags]: Flags[K];
};
```

✅ Dynamic remapping of keys

✅ Great with utility types like `Partial<T>`, `Readonly<T>`, `Record<K, V>`

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Overusing index signatures	Define specific keys when possible for better DX
Mixing fixed + dynamic keys wrongly	Ensure types match
Forgetting `readonly` for constants	Bugs when mutated
Assuming optional props are always defined	Use safe access (`?.`) or fallback logic
Mutating input objects in place	Use readonly types or immutability helpers

---

✅ Real-World Use: API Response Typing

```
type ApiResponse<T> = {
  success: boolean;
  data?: T;
  error?: string;
};
```

```
function handleResponse<T>(res: ApiResponse<T>) {
  if (res.success && res.data) {
    // Use res.data
  }
}
```

✅ Optional + readonly types are useful for **API layers**, **forms**, and **config**

🧪 Summary Cheatsheet

Feature	Syntax	Use For
Strict object	`type A = { x: number }`	Precise object shapes
Optional	`name?: string`	Partial or undefined props

Feature	Syntax	Use For
Readonly	`readonly x: string`	Immutable fields
Index sig	`[key: string]: string`	Maps, lookup tables
Structural typing	Duck typing	Loose compatibility
Assignment rules	Direct = strict	Extra keys must match

▼ 🔁 Union & Intersection Types

✅ 1. Union Types – "This OR That"

Use when a value **can be one of several types**.

```
type Status = "loading" | "success" | "error";

let apiStatus: Status = "loading"; // ✅
```

```
type Id = number | string;

let userId: Id = 42;
userId = "abc"; // ✅ both are valid
```

✅ Best for:

- **Status enums** (`"pending" | "done"`)
- **Flexible inputs** (`number | string`)
- **Polymorphic component props**

🧠 Real-World Usage Example

```
type ButtonSize = "sm" | "md" | "lg";

interface ButtonProps {
  size: ButtonSize;
```

```
  disabled?: boolean;
}
```

🔎 2. Narrowing Union Types (Type Guards)

You must **narrow** before using union-specific methods.

```
function printId(id: number | string) {
  if (typeof id === "string") {
    console.log(id.toUpperCase()); // ✅ safe
  } else {
    console.log(id.toFixed(2)); // ✅ safe
  }
}
```

🧰 Common Narrowing Tools:

Method	Example
`typeof`	`typeof id === "string"`
`instanceof`	`value instanceof Date`
`'in'` keyword	`'price' in product`
Custom predicates	`isUser(val): val is User`

✅ Discriminated Unions (Pattern Matching)

```
type Circle = { kind: "circle"; radius: number };
type Square = { kind: "square"; side: number };

type Shape = Circle | Square;

function getArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
```

```
      return Math.PI * shape.radius ** 2;
    case "square":
      return shape.side * shape.side;
  }
}
```

✅ This is extremely common in React components, reducers, API response types

---

⚠️ Common Pitfall

```
function getLength(val: string | string[]) {
  return val.length; // ❌ Error — not narrowed
}
```

✅ Fix:

```
if (typeof val === "string") return val.length;
return val.length;
```

🔗 3. Intersection Types – “This AND That”

Use when an object must **satisfy multiple types**.

```
type User = { id: number; name: string };
type Admin = { role: "admin"; accessLevel: number };

type AdminUser = User & Admin;

const admin: AdminUser = {
  id: 1,
  name: "Sahil",
  role: "admin",
```

```
  accessLevel: 5,
};
```

✅ Combine shared models (`User & Permissions`)

✅ Build **extendable, composable** types in large apps

🔶 Interface Extension = Intersection

```
interface A { a: string }
interface B { b: string }

type AB = A & B;
```

This behaves the same as:

```
interface AB extends A, B {}
```

🧠 Practical Use: Combining Props

```
type BaseProps = { className?: string };
type ButtonProps = { onClick: () => void };

type FullProps = BaseProps & ButtonProps;
```

✅ Great for component props composition

⚠️ Pitfall with Conflicting Types

```
type A = { status: string };
type B = { status: number };

// ❌ Conflict: can't be both string AND number
type AB = A & B; // Error or `never` type
```

> 🔥 Intersection of incompatible types = unusable type

🧪 Summary Cheatsheet

Type	Syntax	Meaning
Union	`A	B`
Intersection	`A & B`	A **and** B
Narrowing	`typeof`, `in`, `instanceof`	Identify exact type at runtime
Discriminated Union	`type` with `"kind"` property	Switch-safe union pattern

---

✅ When to Use

Use Case	Use
Status handling	Union `"success"
Polymorphic props	Union
Merging object shapes	Intersection
Component props reuse	Intersection
API error/result responses	Discriminated unions
User role modeling	Intersection of `User & Admin & Editor`

🧠 Pro Tip: Use with `Partial<T>`, `Record<K, V>`, `Exclude<T, U>`

```
type AdminUser = User & { role: "admin" };
type MaybeId = string | null | undefined;
```

▼ 🚀 Generics (Production Gold)

🧠 What are Generics?

> Generics let you write functions, types, or components that work with any type, but stay type-safe.

You define the type **later**, when you **use** it.

```
function identity<T>(arg: T): T {
  return arg;
}

identity<string>("hello"); // ✅ string
identity<number>(42);      // ✅ number
```

✅ 1. Generic Functions

🔷 Basic Example

```
function wrapInArray<T>(item: T): T[] {
  return [item];
}
```

Usage:

```
const result = wrapInArray("hello"); // string[]
```

> ✅ TS infers T = string from the call
>
> 💡 Use generics instead of `any` to retain type context

✅ 2. Generic Arrow Functions

```
const logAndReturn = <T>(value: T): T => {
  console.log(value);
  return value;
};
```

> ✅ Same as function version — use whichever fits your codebase style.

✅ 3. Generic Interfaces & Types

Used in React props, API layers, and reusable data types.

```
interface ApiResponse<T> {
  success: boolean;
  data: T;
  error?: string;
}

const userRes: ApiResponse<{ id: number; name: string }> = {
  success: true,
  data: { id: 1, name: "Sahil" },
};
```

> ✅ Now you can reuse ApiResponse<T> across user, post, auth, etc.

📦 4. Generics with Arrays & Promises

```
function firstItem<T>(arr: T[]): T {
  return arr[0];
}

async function fetchData<T>(): Promise<T> {
  const res = await fetch("/api/data");
  return res.json();
}
```

✅ `Promise<T>` is one of the most common built-in generic types.

🔐 5. Constraints in Generics

Restrict the kind of types that can be passed:

```
function getId<T extends { id: string }>(item: T): string {
  return item.id;
}
```

✅ Ensures `T` **must** have an `id: string` field.

```
getId({ id: "123", name: "Shakil" }); // ✅ OK
getId({ name: "Fail" }); // ❌ Error: missing `id`
```

🔄 6. Default Generic Types

You can set defaults if type isn't provided:

```
type ApiResult<T = unknown> = {
  data: T;
  error?: string;
};

const result: ApiResult = {
  data: { msg: "Works!" }, // data: unknown
};
```

✅ Use defaults when consumers may not always pass a type

🧩 7. Generic Component Props (React Example)

```
type ListProps<T> = {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
};

function List<T>({ items, renderItem }: ListProps<T>) {
```

```
  return <ul>{items.map(renderItem)}</ul>;
}
```

Usage:

```
<List
  items={[{ id: 1, name: "Alice" }]}
  renderItem={(user) => <li key={user.id}>{user.name}</li>}
/>
```

✅ Strongly typed, flexible, reusable UI logic

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Using `any` when generics apply	Define `<T>` and infer
Forgetting constraints (`extends`)	Results in runtime undefined issues
Overcomplicating generics	Simpler union or function overload may work
Forgetting to pass type args	Set defaults (`<T = unknown>`)

✅ Real-World Usage Patterns

Scenario	Use
Reusable data model	`type ApiResponse<T>`
Flexible components	`<T>(props: Props<T>)`
Event handler typing	`React.MouseEvent<T>`
Redux/RTK generic actions	`ActionCreator<T>`
Form builders	`<Field<T>>`

🧪 Summary Cheatsheet

Pattern	Example
Generic function	`<T>(val: T): T`
Arrow version	`const fn = <T>(x: T) => ...`
Generic type	`type Wrapper<T> = { data: T }`
Constraint	`<T extends { id: string }>`
Default	`<T = unknown>`

🚨 Remember

- Generics are **not magic** — they simply **defer the type decision**
- Avoid `any`; **use** `<T>` **instead** wherever possible
- Use **constraints (** `extends` **)** to guide safe usage

▼ 🎯 Advanced Types

✅ 1. Type Inference

Let TypeScript **automatically infer** types when obvious.

```
const name = "Sahil"; // type: string (inferred)
let age = 25;         // type: number
```

✅ Preferred unless:

- You're authoring a reusable function
- You need explicit type safety on public API

> 🧠 Inference reduces boilerplate, but never at the cost of clarity

---

✅ 2. Literal Types

```
type Status = "loading" | "success" | "error";
```

```
let state: Status = "loading";
```

✅ Creates a narrow type

✅ Use with unions, enums, switch statements

```
const role = "admin" as const; // role: "admin" (not string)
```

🔑 3. `keyof` Operator

Creates a union of all property names of a type.

```
type User = { id: number; name: string };
type UserKeys = keyof User; // "id" | "name"
```

✅ Often used with generics or mapped types

🔍 4. `typeof` Operator

Used to extract types **from variables**:

```
const config = {
  port: 3000,
  baseUrl: "https://api.com",
};

type Config = typeof config;
```

✅ Use this to avoid repeating types for constants or functions.

🧠 5. Conditional Types

```
type IsString<T> = T extends string ? true : false;
```

```
type A = IsString<"hello">; // true
type B = IsString<number>;  // false
```

✅ Enables **type logic** (like ternary)

✅ Used heavily in advanced libraries (Zod, Formik, etc.)

🔁 6. Mapped Types

Create new types from existing ones.

```
type User = {
  name: string;
  age: number;
};

type ReadonlyUser = {
  readonly [K in keyof User]: User[K];
};
```

✅ Same as `Readonly<User>`

✅ Foundation of all utility types

🧰 7. Built-in Utility Types (with 🔥 Real Use Cases)

🔷 `Partial<T>`

Makes all fields optional

```
type User = { id: number; name: string };
type UserUpdate = Partial<User>; // { id?: number; name?: string }
```

✅ Great for PATCH APIs or form drafts

🔷 `Required<T>`

Opposite of `Partial`, makes all fields mandatory

```
type User = { id?: number; name?: string };
type StrictUser = Required<User>; // { id: number; name: string }
```

🔷 `Readonly<T>`

Immutable fields

```
type User = { id: number; name: string };
type LockedUser = Readonly<User>; // can't mutate
```

✅ Protects against accidental mutation

🔷 `Record<K, V>`

Creates an object type with specific keys and value types

```
type Theme = "light" | "dark";

const themeMap: Record<Theme, string> = {
  light: "#fff",
  dark: "#000",
};
```

✅ Great for lookup tables or enums

🔷 `Pick<T, K>`

Select specific keys

```
type User = { id: number; name: string; age: number };
type PublicUser = Pick<User, "id" | "name">;
```

🔷 `Omit<T, K>`

Remove specific keys

```
type PrivateUser = Omit<User, "age">;
```

🔷 `Exclude<T, U>`

Remove members of union `U` from `T`

```
type Status = "loading" | "success" | "error";
type NonLoading = Exclude<Status, "loading">; // "success" | "error"
```

🔷 `Extract<T, U>`

Opposite of `Exclude`

```
type Response = "success" | "error";
type OnlySuccess = Extract<Response, "success">; // "success"
```

🔷 `NonNullable<T>`

Remove `null` and `undefined` from type

```
type MaybeUser = string | null | undefined;
type DefiniteUser = NonNullable<MaybeUser>; // string
```

🔷 `ReturnType<T>`

Get return type of a function

```
function getUser() {
  return { id: 1, name: "Shakil" };
}

type User = ReturnType<typeof getUser>;
```

🔷 `Parameters<T>`

Get parameter types of a function as a tuple

```
function sendMessage(to: string, msg: string) {}

type SendArgs = Parameters<typeof sendMessage>; // [string, string]
```

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Misusing `typeof` in value context	Only use it in **type declarations**
Overusing utility types	Create your own with mapped/conditional types
Forgetting null/undefined checks	Use `NonNullable<T>`
Confusing `Pick` vs `Omit`	Think: Pick = **keep**, Omit = **remove**

🔥 Real Use: Composing API & Form Types

```
type User = {
  id: string;
  name: string;
  email: string;
};

type UserForm = Omit<User, "id">; // for form
type UserPreview = Pick<User, "name" | "email">;
```

✅ You can reuse core models and transform them easily

🧪 Summary Cheatsheet

Utility	Purpose
`Partial<T>`	All optional

Utility	Purpose
`Required<T>`	All required
`Readonly<T>`	Immutable
`Record<K, V>`	Key-value map
`Pick<T, K>`	Select keys
`Omit<T, K>`	Remove keys
`Exclude<T, U>`	Remove types
`Extract<T, U>`	Keep matching types
`NonNullable<T>`	Remove `null` / `undefined`
`ReturnType<T>`	Get return type
`Parameters<T>`	Get parameter types

▼ 🔍 Type Assertions & Casting

TypeScript sometimes can't infer types with 100% confidence. Type **assertions** let you tell the compiler:

> "Trust me, I know what I'm doing."

But with **great power comes great risk** — so use sparingly and intentionally.

✅ 1. `as` Keyword — Type Assertion

For **casting** a value from one type to another:

```
const input = document.querySelector("input") as HTMLInputElement;

console.log(input.value); // ✅ now TS knows it's not null and has `.value`
```

🔷 Generic Usage

```
const someVal: unknown = "Shakil" as string;
```

✅ Used when TS infers a broader type like `unknown` or `HTMLElement`, and you know more specific details.

⚠️ Double Cast (rare, use with caution)

```
const json = "{}";
const parsed = JSON.parse(json) as unknown as User;
```

✅ Used when converting from `any/unknown` → validated domain type

❌ Do this only after proper runtime validation (e.g. Zod, Yup)

❗ 2. Non-null Assertion (`!` operator)

Used to tell TypeScript:

> "This value will definitely not be null or undefined."

```
const el = document.getElementById("main")!;
el.innerHTML = "Loaded";
```

✅ Useful in React refs, DOM access, router context

⚠️ Why Dangerous?

Because it silences **real null/undefined safety**. If wrong, it causes **runtime crashes**.

✅ Use Cases in React

🔷 1. Ref in function component

```
const inputRef = useRef<HTMLInputElement>(null);

useEffect(() => {
```

```
  inputRef.current!.focus(); // ! used to tell TS it's definitely set
}, []);
```

🔷 2. DOM access in effects

```
const node = document.querySelector(".hero") as HTMLElement;
node.style.opacity = "1";
```

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Using `as` to fix type errors blindly	Fix root cause or use type guards
Overusing `!` for convenience	Use optional chaining (`?.`) or conditionals
Skipping runtime checks	Combine with Zod/Yup for safe API parsing
Using `as` to bypass `null` checks	Prefer `if (ref?.current)` over `ref!.current`

🧪 Real-World Tips

Use Case	Assertion
DOM elements	`as HTMLDivElement`
Refs	`ref.current!`
JSON parsing	`JSON.parse(...) as MyType`
Third-party data	Combine `as` with validation (`z.infer<typeof schema>`)
Overwriting broad types	`value as SpecificType`

🧠 Summary Cheatsheet

Feature	Syntax	Notes
Type Assertion	`val as Type`	Tell TS a better type
Non-null Assertion	`val!`	Remove `null/undefined`
Double Cast	`as unknown as T`	From `any/unknown` to known

Feature	Syntax	Notes
Safe Alternative	Type guards, runtime checks	Better for long-term safety

✅ Final Advice

- ✅ Use `as` in **interop zones** (DOM, refs, JSON)
- ⚠️ Avoid `!` unless you're 100% sure the value exists
- ❌ Never use assertions to “fix” compile errors without understanding them

▼ ⚙️ Modules & Namespaces

✅ 1. Import/Export Syntax with Types

TypeScript supports **ES Modules** syntax. You can export **values**, **types**, or both.

🔷 Exporting Values

```
export const API_URL = "https://api.example.com";

export function sum(a: number, b: number): number {
  return a + b;
}
```

🔷 Importing Values

```
import { API_URL, sum } from "./utils";
```

🔷 Exporting Types

```
export type User = {
  id: number;
```

```
  name: string;
};
```

🔷 Importing Types (TS-only import)

```
import type { User } from "./types";
```

✅ `import type` ensures that it's **erased at compile time** — no JS output

---

⚡ Real-World Tip

> 🔥 Use import type for type-only imports when using bundlers like Vite or Webpack for better tree-shaking and faster builds.

🔷 Default Exports (Not recommended in TS-heavy teams)

```
// ❌ Often discouraged due to refactor bugs
export default function App() {}
```

```
import App from "./App"; // no curly braces
```

✅ Prefer **named exports** for better auto-complete & maintainability

---

📁 2. Declaring Module Types (e.g. SVG, CSS Modules, JSON)

TypeScript doesn't know about **non-code files** (e.g., `.svg`, `.css`)

You must declare them manually using a **declaration file**:

✅ SVG Module Declaration

```
// types/svg.d.ts
declare module "*.svg" {
```

```
  const content: string;
  export default content;
}
```

✅ CSS Modules Declaration

```
// types/cssModules.d.ts
declare module "*.module.css" {
  const classes: { [key: string]: string };
  export default classes;
}
```

✅ JSON Modules (usually auto-inferred)

```
declare module "*.json" {
  const value: any;
  export default value;
}
```

🧠 Best Practice: Keep custom declarations in `types/` folder and add this to `tsconfig.json`:

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "~types/*": ["types/*"]
    }
  },
  "include": ["src", "types"]
}
```

🚀 Bonus: Exporting Reusable Type & Utility Modules

```
// src/types/index.ts
export type { User, Post } from "./models";
export type { Theme } from "./theme";
```

```
// src/utils/index.ts
export * from "./date";
export * from "./math";
```

✅ Now you can import from a single source:

```
import { formatDate, sum } from "@/utils";
```

🧩 3. What About Namespaces?

Namespaces are **legacy** (pre-ESM era). Avoid them unless you're working in:

- Older JS codebases (non-module)
- Library internal typings (e.g., `.d.ts` for polyfills)

```
namespace MathUtils {
  export const PI = 3.14;
  export function square(n: number) {
    return n * n;
  }
}

MathUtils.square(2); // ✅
```

> ❌ Don't mix namespace with modules — it's not compatible with ES modules
>
> ✅ Prefer **modules over namespaces** in all modern projects

---

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Using `import` without `type` for types	Use `import type` when needed
Forgetting to declare file modules	Add `*.d.ts` for CSS, SVG, JSON
Default exports everywhere	Prefer named exports
Using `namespace` in ESM projects	Use modules instead

🧪 Summary Cheatsheet

Feature	Syntax	Use
Named export	`export { a, b }`	Recommended
Type-only import	`import type { T }`	For cleaner builds
CSS/SVG import	Declare module in `.d.ts`	
Avoid `namespace`	Prefer `export` + `import`	

✅ Real Usage: SVG Import in React

```
import Logo from "./logo.svg"; // after declaring module "*.svg"

<img src={Logo} alt="Logo" />
```

▼ 🧩 Working with Third-Party Libraries

✅ 1. Installing `@types` Packages

Many popular JS libraries don't include their own types.

TypeScript uses a community-maintained repo called **DefinitelyTyped** for this.

```
npm install --save-dev @types/lodash
```

🔷 Examples:

Library	Types Package
`lodash`	`@types/lodash`
`moment`	`@types/moment`
`express`	`@types/express`
`react-router-dom`	`@types/react-router-dom` (for v5 only)

✅ TypeScript will now understand the API and give you autocomplete + error-checking.

🧠 Rule of Thumb:

> If the library doesn't export types, try:
>
> `npm i @types/<lib-name>` (as a devDependency)

📦 2. Using `DefinitelyTyped`

- It's a huge GitHub repo of `.d.ts` files
- Automatically distributed via npm under the `@types/` scope
- You don't need to clone or star it, just install relevant packages
- Used by **99% of the open-source ecosystem** (when types aren't bundled)

🔧 3. Writing Basic Custom Type Declarations

Some libraries, assets, or internal code don't ship with types.

You can **manually declare them**:

✅ `declare module` (basic example)

```
// types/custom-module.d.ts
declare module "untyped-lib" {
```

```
  export function doSomething(input: string): number;
}
```

✅ Now you can import & use `untyped-lib` without TS errors.

✅ Declaring for Assets (SVG, CSS)

```
declare module "*.svg" {
  const content: string;
  export default content;
}

declare module "*.module.css" {
  const classes: { [key: string]: string };
  export default classes;
}
```

> ✅ Add these in a types/ folder and reference in tsconfig.json via "include": ["src", "types"]

✅ Fallback: Declare all as `any` (temporary)

```
declare module "some-broken-lib";
```

⛔ Dangerous — use only as a last resort when you just want it to compile.

🧪 Real Example: Untyped API Client

```
// types/fake-api-client.d.ts
declare module "my-api-client" {
  export type User = { id: number; name: string };
  export function getUsers(): Promise<User[]>;
}
```

✅ Helps bridge the gap until the library adds types or you migrate to a typed version.

💡 Bonus: Declare Global Types

```
// types/global.d.ts
interface Window {
  __APP_VERSION__: string;
}
```

✅ Now you can use `window.__APP_VERSION__` safely anywhere.

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Skipping types for popular packages	Always check `@types/<name>`
Using `any` everywhere	Create `.d.ts` files with minimal shapes
Not including custom types in tsconfig	Use `"include": ["src", "types"]`
Ignoring type errors from 3rd party libs	Patch types via `declare module`

✅ Summary Cheatsheet

Task	Tool/Approach
Install missing types	`npm i -D @types/lodash`
Asset modules	`declare module "*.svg"`
Untyped JS library	`declare module "lib-name"`
Add global types	`interface Window {}`
Reference custom types	Add to `tsconfig.json` includes

🚀 Real Usage with Vite + React

```
// vite-env.d.ts
/// <reference types="vite/client" />

declare module "*.svg" {
  const content: string;
  export default content;
}
```

▼ ⚛️ TypeScript with React

✅ 1. Typing Functional Components

🔷 With `FC` (FunctionComponent) – Deprecated in large teams

```
import React, { FC } from "react";

type Props = { title: string };

const Header: FC<Props> = ({ title, children }) => (
  <h1>{title} {children}</h1>
);
```

⚠️ **Avoid using** `FC` for these reasons:

- `children` is **automatically included**, even when not required
- `FC` doesn't infer generic types well
- `defaultProps` can break silently

✅ Preferred: Normal Function Typing

```
type Props = { title: string };

function Header({ title }: Props) {
```

```
  return <h1>{title}</h1>;
}
```

✅ Better autocompletion

✅ Manual control over `children`

👶 2. Typing Props + Children

```
type Props = {
  children?: React.ReactNode;
  label: string;
};

const Button = ({ children, label }: Props) => (
  <button>{label} {children}</button>
);
```

⚙️ 3. Typing `useState`

```
const [count, setCount] = useState<number>(0);
```

If TypeScript can infer the type from the initial value, you **don't need to manually add it**:

```
const [text, setText] = useState("hello"); // inferred as string
```

⏱️ 4. Typing `useEffect`

```
useEffect(() => {
  console.log("mounted");

  return () => {
```

```
    console.log("cleanup");
  };
}, []);
```

✅ No extra typing required — TS infers everything automatically.

But if you're using async inside:

```
useEffect(() => {
  const fetchData = async () => {
    const res = await fetch("/api");
    const data = await res.json();
  };

  fetchData();
}, []);
```

✅ Avoid making the `useEffect` function itself `async`

🖱️ 5. Typing Events

🔷 Mouse Events

```
const handleClick = (e: React.MouseEvent<HTMLButtonElement>) => {
  console.log(e.currentTarget);
};
```

🔷 Input Change

```
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  console.log(e.target.value);
};
```

🔷 Form Submit

```
const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
};
```

📦 6. Typing `useRef`

🔷 DOM Elements

```
const divRef = useRef<HTMLDivElement>(null);

useEffect(() => {
  divRef.current?.focus(); // safe with optional chaining
}, []);
```

🔷 Mutable Values (not tied to DOM)

```
const countRef = useRef<number>(0);
```

✅ Use `useRef` when you want to **persist value** across renders **without triggering re-render**.

🧠 7. Typing Context API

🔷 Create Context

```
type Theme = "light" | "dark";

type ThemeContextType = {
  theme: Theme;
  toggleTheme: () => void;
};
```

```
const ThemeContext = createContext<ThemeContextType | null>(null);
```

🔷 Provide Context

```
const ThemeProvider = ({ children }: { children: React.ReactNode }) => {
  const [theme, setTheme] = useState<Theme>("light");

  const toggleTheme = () => {
    setTheme(t => (t === "light" ? "dark" : "light"));
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

🔷 Consume Context

```
const ctx = useContext(ThemeContext);
if (!ctx) throw new Error("ThemeContext not found");

const { theme } = ctx;
```

✅ Always handle the `null` case or provide a default.

🪝 8. Typing Custom Hooks

```
function useToggle(initial: boolean): [boolean, () => void] {
  const [state, setState] = useState(initial);
  const toggle = () => setState(s => !s);
```

```
  return [state, toggle];
}
```

✅ Always specify return types for better DX in consumer components.

🧭 9. Typing with React Router v6+

🔷 `useNavigate`, `useParams`, `useLocation`

```
import { useNavigate, useParams, useLocation } from "react-router-dom";

const navigate = useNavigate();
const params = useParams<{ id: string }>(); // ⚠️ might be undefined

const handleClick = () => navigate("/home");
```

🔷 Route Params are optional by default

```
const { id } = useParams<{ id: string }>();

if (!id) return <div>Loading...</div>;
```

⚠️ Production Warnings

Pitfall	Solution
Overusing `FC` for components	Use normal function typing
Forgetting optional chaining on `ref.current`	Always use `ref.current?.`
Not handling `null` from `createContext<T	null>`
Typing `useEffect` as async	Use inner async function instead
`useParams` value is `string	undefined`

🧪 Summary Cheatsheet

Feature	Typing
Functional Component	`function Comp(props: Props)`
`children`	`React.ReactNode`
`useState`	`useState<Type>()`
`useEffect`	No manual typing
Events	`React.MouseEvent`, `ChangeEvent`, `FormEvent`
`useRef`	`useRef<HTMLDivElement>(null)`
Context	`createContext<Type
Router params	`useParams<{ id: string }>()`

▼ 🔥 TypeScript with Next.js

✅ 1. Typing `getStaticProps`

When using `getStaticProps`, TypeScript can infer props automatically — **but explicit return type is preferred** for reusability and DX.

```
import { GetStaticProps } from 'next'

type Props = {
  posts: { id: string; title: string }[]
}

export const getStaticProps: GetStaticProps<Props> = async () => {
  const posts = await fetchPosts() // returns Post[]

  return {
    props: {
      posts,
    },
  }
}
```

✅ Now your `HomePage` component receives `posts: Post[]` safely.

✅ 2. Typing `getServerSideProps`

Same pattern, just use `GetServerSideProps<T>`:

```
import { GetServerSideProps } from 'next'

type Props = {
  user: { id: number; name: string }
}

export const getServerSideProps: GetServerSideProps<Props> = async (c
ontext) => {
  const user = await fetchUserById(1)

  return {
    props: { user },
  }
}
```

✅ All context objects (`params`, `req`, `res`, etc.) are automatically typed.

🧩 3. Typing `NextPage`

Every Next.js page can be typed using `NextPage<Props>`:

```
import type { NextPage } from 'next'

type Props = {
  user: { name: string }
}

const ProfilePage: NextPage<Props> = ({ user }) => {
  return <h1>Hello {user.name}</h1>
}
```

🛠️ 4. Typing API Routes

API routes in `pages/api/` should type their **req/res objects** using `NextApiRequest` and `NextApiResponse`.

```
import type { NextApiRequest, NextApiResponse } from 'next'

type Data = {
  success: boolean
  message: string
}

export default function handler(req: NextApiRequest, res: NextApiRespons
e<Data>) {
  res.status(200).json({ success: true, message: 'Hello from API' })
}
```

✅ Response will only accept the shape of `Data`

🧪 5. Dynamic Params + `getStaticPaths`

```
export const getStaticPaths: GetStaticPaths = async () => {
  const paths = [{ params: { slug: "post-1" } }]

  return {
    paths,
    fallback: false,
  }
}
```

And type `params` in `getStaticProps`:

```
export const getStaticProps: GetStaticProps = async (context) => {
  const { slug } = context.params as { slug: string }
```

```
  const post = await getPost(slug)
  return {
    props: { post },
  }
}
```

> ✅ Use as assertion because context.params is typed as ParsedUrlQuery which returns string | string[] | undefined

---

⚡ Bonus: Type Utilities from `next` and `next-auth`

- `NextPage<Props>` – Page components
- `GetStaticProps`, `GetServerSideProps`, `GetStaticPaths`
- `NextApiRequest`, `NextApiResponse`
- `InferGetStaticPropsType<typeof getStaticProps>` – extract props

```
type Props = InferGetStaticPropsType<typeof getStaticProps>
```

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Forgetting `as` on `params`	`const { id } = params as { id: string }`
Loose API typing	Use `NextApiRequest`, `NextApiResponse<T>`
Untyped props in pages	Use `NextPage<Props>`
Forgetting return types in `getStaticProps`	Add `: GetStaticProps<T>` manually

✅ Summary Cheatsheet

Task	Typing
Static Props	`GetStaticProps<T>`
Server Props	`GetServerSideProps<T>`

Task	Typing
Page Component	`NextPage<T>`
API Handler	`(req: NextApiRequest, res: NextApiResponse<T>)`
Inferred Props	`InferGetStaticPropsType<typeof getStaticProps>`
Params	`context.params as { id: string }`

▼ ⚙️ TypeScript with Node.js

✅ 1. Typing Express Requests & Responses

Start by installing types:

```
npm install express
npm install --save-dev @types/express
```

Then use the built-in typings:

```
import express, { Request, Response } from "express";

const app = express();

app.get("/ping", (req: Request, res: Response) => {
  res.json({ msg: "pong" });
});
```

🔷 Strongly Typed Route Params

```
app.get("/user/:id", (req: Request<{ id: string }>, res: Response) => {
  const userId = req.params.id;
  res.send(`User ID: ${userId}`);
});
```

🔷 Strongly Typed Query Params

```
app.get("/search", (req: Request<{}, {}, {}, { q: string }>, res: Response) ⇒
{
  const query = req.query.q;
  res.send(`Search: ${query}`);
});
```

🧾 2. Custom Type Declarations for Request Body

Define input type separately for cleaner code:

```
type LoginBody = {
  email: string;
  password: string;
};

app.post("/login", (req: Request<{}, {}, LoginBody>, res: Response) ⇒ {
  const { email, password } = req.body;
  res.json({ success: true });
});
```

✅ Now you get full type safety and IntelliSense for request bodies!

🧠 3. Typing `next()` and Custom Middleware

```
import { NextFunction } from "express";

function logger(req: Request, res: Response, next: NextFunction) {
  console.log(`[${req.method}] ${req.url}`);
  next();
}
```

🧬 4. Typing MongoDB Models with Mongoose

Install:

```
npm install mongoose
npm install --save-dev @types/mongoose
```

🔷 Define Interface for Document

```
import mongoose, { Document, Schema, model } from "mongoose";

interface IUser extends Document {
  name: string;
  email: string;
  isAdmin: boolean;
}
```

🔷 Create Mongoose Schema + Model

```
const userSchema = new Schema<IUser>({
  name: { type: String, required: true },
  email: { type: String, required: true },
  isAdmin: { type: Boolean, default: false },
});

const UserModel = model<IUser>("User", userSchema);
```

✅ Now TS knows the shape of `UserModel`:

```
const user = await UserModel.findById("abc123");

if (user) {
  user.name; // ✅ string
  user.isAdmin; // ✅ boolean
}
```

🔄 With `Lean()` Queries

Lean queries remove Mongoose methods from documents. Use a generic fallback type:

```
const result = await UserModel.find().lean<IUser>().exec();
```

✅ Now `result` is a plain object array, but still typed

🧪 Bonus: Extending `mongoose.Model` with statics

```
interface IUserModel extends mongoose.Model<IUser> {
  findByEmail(email: string): Promise<IUser | null>;
}

const schema = new mongoose.Schema<IUser, IUserModel>({
  name: String,
  email: String,
});

schema.statics.findByEmail = function (email: string) {
  return this.findOne({ email });
};

const User = mongoose.model<IUser, IUserModel>("User", schema);
```

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Typing only `req.body` as `any`	Use `Request<{}, {}, YourType>`
Forgetting query/params types	Add `<Params, ResBody, ReqBody, ReqQuery>`
Using `.lean()` without type	Add `<Type>` to `.lean<Type>()`

❌ Pitfall	✅ Fix
Using `Document` alone for Mongoose models	Use `interface` + `Schema<IType>`
Not typing middleware `next`	Import and type as `NextFunction`

✅ Summary Cheatsheet

Use Case	Syntax
Express Request + Params	`Request<{ id: string }>`
Request Body	`Request<{}, {}, YourBody>`
Query Params	`Request<{}, {}, {}, YourQuery>`
Response Type	`Response<YourRes>`
Mongoose Document	`interface IUser extends Document {}`
Mongoose Schema	`new Schema<IUser>()`
Middleware	`(req: Request, res: Response, next: NextFunction)`

▼ 💡 Error Handling & Best Practices

✅ 1. Avoiding the `any` Type — and When It's Okay

❌ Bad:

```
function fetchData(): any {
  return axios.get("/api");
}
```

✅ Good:

```
function fetchData(): Promise<User[]> {
  return axios.get("/api/users").then(res => res.data);
}
```

🔷 When `any` is Okay (Temporarily)

- Rapid prototyping (short-term)
- Working with untyped 3rd-party data (until types are defined)
- Testing/isolating broken types temporarily (add `TODO`)

```
// TODO: fix this any
const res: any = someBrokenLib();
```

🔐 2. Using `unknown` Instead of `any`

- `unknown` is **type-safe**: You must **narrow** it before use
- Think of it as a **safer version of any**

```
function parse(input: unknown) {
  if (typeof input === "string") {
    return input.toUpperCase();
  }
  return null;
}
```

✅ Prevents accidental access to unsafe fields

❌ `any` lets you do anything, even invalid ops

🧠 3. Enable `strict` Mode in `tsconfig.json`

```
{
  "compilerOptions": {
    "strict": true
  }
}
```

Enables:

Flag	Benefit
`noImplicitAny`	Disallows silent `any`
`strictNullChecks`	Forces safe handling of `null` / `undefined`
`strictFunctionTypes`	Ensures safe callbacks
`strictBindCallApply`	Prevents broken `bind` usage
`alwaysStrict`	Forces `use strict`

✅ Forces you to handle edge cases explicitly

🔁 4. Keeping Types DRY & Modular

❌ Bad:

```
function login(req: Request<{}, {}, { email: string; password: string }>) {}
```

✅ Good:

```
type LoginBody = {
  email: string;
  password: string;
};

function login(req: Request<{}, {}, LoginBody>) {}
```

✅ Define types once, reuse everywhere (frontend + backend)

🔷 Shared Types Example

```
// /types/user.ts
export type User = {
  id: string;
  name: string;
};
```

```
// ✅ use in both backend & frontend
```

✅ Use path aliases in `tsconfig.json` for clean imports:

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@types/*": ["types/*"]
    }
  }
}
```

❗ 5. Common TS Errors & Their Meaning

Error	Meaning	Fix
`Object is possibly 'undefined'`	You're not checking `undefined`	Add `?.` or an `if` check
`Type 'X' is not assignable to type 'Y'`	Wrong type being passed	Fix the argument or expected type
`Parameter 'x' implicitly has an 'any' type`	Missing type	Add explicit type or enable `noImplicitAny`
`Cannot read property 'x' of undefined`	Runtime error due to missing checks	Use optional chaining or fallback
`Argument of type 'string' is not assignable to parameter of type 'number'`	TS caught incorrect param type	Convert or validate before passing

---

✅ 6. Bonus Tips

🔷 Use `NonNullable<Type>`

```
type User = { name: string | null };

const safeUser: NonNullable<User['name']> = "Shakil"; // ensures no null
```

🔷 Never overuse `!` (non-null assertion)

```
inputRef.current!.focus(); // ✅ only if you're sure
```

Instead:

```
inputRef.current?.focus(); // ✅ safer
```

📦 7. Summary Cheatsheet

Principle	Action
Avoid `any`	Use `unknown`, or type properly
Enable `strict`	Improves long-term safety
DRY types	Export types to shared files
Handle `undefined`	Use optional chaining / fallback
Learn TS errors	They're actually helpful guides

🧠 Golden Rule

> "The goal of TypeScript isn't to write fewer bugs — it's to write fewer surprises."

▼ 🚨 Real Production Patterns

✅ 1. Declaring Global Types

Global types are helpful for:

- `window` extensions (e.g. analytics, versioning)
- custom environment variables (`process.env`)
- auto-imported types used across many files (like `User`)

📁 Where to Put Global Types

```
// types/global.d.ts
declare global {
  interface Window {
    __APP_VERSION__: string;
  }

  namespace NodeJS {
    interface ProcessEnv {
      NODE_ENV: "development" | "production" | "test";
      NEXT_PUBLIC_API_URL: string;
    }
  }
}

export {}; // 👈 Ensures file is treated as a module
```

✅ Include `types/` Folder in `tsconfig.json`

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@types/*": ["types/*"]
    }
  },
  "include": ["src", "types"]
}
```

✅ You can now use `window.__APP_VERSION__` anywhere, fully typed.

📦 2. Creating `types/` Folder with Barrel File

In large apps, it's common to structure types like:

```
types/
├── api.d.ts
├── models/
│   ├── user.ts
│   ├── product.ts
├── global.d.ts
└── index.ts 👈 barrel file
```

✅ Example: `types/models/user.ts`

```
export type User = {
  id: string;
  name: string;
  email: string;
};
```

✅ `types/index.ts`

```
export * from "./models/user";
export * from "./models/product";
export * from "./api";
```

✅ Usage:

```
import { User } from "@types"; // clean & central
```

🤔 3. `type` vs `interface`: Which One & When?

Both are used to **describe shapes**, but their roles vary in large-scale apps.

🛠️ Use `type` When:

- Creating **unions, intersections, tuples**
- You need to **compose** complex types
- Avoiding accidental extension

```
type APIResponse = { status: number } | { error: string };

type User = {
  name: string;
} & {
  id: number;
};
```

🧩 Use `interface` When:

- Defining objects that **can be extended**
- Creating **class contracts**
- Used in **React Props** (optional)

```
interface BaseUser {
  id: string;
}

interface AdminUser extends BaseUser {
  role: "admin";
}
```

✅ Interfaces can be merged (open-ended), types **cannot**:

```
interface Config {
  debug: boolean;
}

interface Config {
  version: string;
}

// Merged → { debug: boolean; version: string }
```

🧠 Recommendation for Large Teams

Use Case	Recommendation
Data models, API shapes	✅ Use `type`
Extensible props or classes	✅ Use `interface`
Complex unions / generics	✅ Use `type`
External lib contracts (like React)	Use what the lib uses

⚠️ Production Warnings

Pitfall	Fix
Scattering types across code	Centralize in `types/` folder
Duplicating types across frontend/backend	Move to shared lib / monorepo
Uncontrolled merging of interfaces	Prefer `type` for critical models
Missing global declarations	Use `global.d.ts` with `declare global`

✅ Summary Cheatsheet

Task	Pattern
Add global env vars	`declare global` inside `global.d.ts`
Centralize all types	Create `types/` folder + `index.ts` barrel

Task	Pattern
Type vs Interface	`type` for unions & final shapes, `interface` for extension
TSConfig paths	`"@types/*": ["types/*"]`
Exporting many types	Barrel pattern: `export * from`

▼ 🛠️ Linting & Formatting

✅ 1. ESLint with TypeScript

📦 Install ESLint and TypeScript plugins

```
npm install -D eslint typescript \
@typescript-eslint/eslint-plugin @typescript-eslint/parser
```

⚙️ Create `.eslintrc.js` (base config)

```
module.exports = {
  parser: '@typescript-eslint/parser',
  parserOptions: {
    ecmaVersion: 'latest',
    sourceType: 'module',
  },
  plugins: ['@typescript-eslint'],
  extends: [
    'eslint:recommended',
    'plugin:@typescript-eslint/recommended', // core TS rules
  ],
  rules: {
    '@typescript-eslint/no-unused-vars': ['warn'],
    '@typescript-eslint/explicit-module-boundary-types': 'off',
  },
};
```

✅ ESLint with React (if using)

```
npm install -D eslint-plugin-react eslint-plugin-react-hooks
```

Add:

```
plugins: ['@typescript-eslint', 'react', 'react-hooks'],
extends: [
  'plugin:react/recommended',
  'plugin:react-hooks/recommended',
  'plugin:@typescript-eslint/recommended',
],
settings: {
  react: { version: 'detect' },
}
```

🎨 2. Prettier Integration

📦 Install Prettier + ESLint bridge plugins

```
npm install -D prettier eslint-config-prettier eslint-plugin-prettier
```

🔧 Update `.eslintrc.js`

```
extends: [
  'plugin:@typescript-eslint/recommended',
  'plugin:prettier/recommended', // 💡 enables prettier plugin + disables co
nflicting rules
],
```

📄 Create `.prettierrc` file

```
{
  "semi": true,
  "singleQuote": true,
  "printWidth": 100,
  "tabWidth": 2,
  "trailingComma": "all"
}
```

🧹 Format files manually

```
npx prettier --write .
```

Or add script:

```
"scripts": {
  "lint": "eslint . --ext .ts,.tsx",
  "format": "prettier --write ."
}
```

🔄 3. Automatically Format on Save (VSCode)

Add to `.vscode/settings.json`:

```
{
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  }
}
```

✅ Now ESLint + Prettier will fix issues on save.

✅ 4. Enforcing Type Checks in CI/CD

Why? Linting ≠ Type checking

TypeScript errors may still compile if not explicitly checked!

🔍 Add `tsc --noEmit` script

```
"scripts": {
  "typecheck": "tsc --noEmit"
}
```

✅ Runs type checking without outputting JS.

🧪 Example GitHub Actions Workflow

```
# .github/workflows/lint-and-typecheck.yml
name: Lint & TypeCheck

on: [push, pull_request]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '20'
      - run: npm ci
      - run: npm run lint
      - run: npm run format
      - run: npm run typecheck
```

✅ Now every PR/commit runs lint, format check, and strict type checking 🚀

⚠️ Production Warnings

❌ Pitfall	✅ Fix
Linting only JS files	Use `eslint . --ext .ts,.tsx`
Skipping type checks in CI	Add `tsc --noEmit` in CI
Conflicting Prettier + ESLint rules	Use `eslint-config-prettier`
Ignoring unused vars	Use `'@typescript-eslint/no-unused-vars': 'warn'`
Not formatting on save	Setup `.vscode/settings.json` properly

✅ Summary Cheatsheet

Tool	Purpose
`eslint`	Linting code for style + errors
`@typescript-eslint/*`	TypeScript-specific linting
`prettier`	Auto-formatting for consistent code style
`eslint-config-prettier`	Disables conflicting ESLint rules
`tsc --noEmit`	Type-only check (use in CI)
`.vscode/settings.json`	Enable auto-format on save